Feed-Forward Neural Networks for Regression and Classification, Compared with Linear and Logistic Regression

Sigurd Riseth
Department of Physics
University of Oslo
Oslo, Norway
sigurri@uio.no

***Abstract*—Neural networks learn their own features, while linear models depend on features chosen by hand, but this flexibility comes at a cost. We asked when the flexibility of a neural network is worth this cost. We implemented feed-forward neural networks from scratch, verified backpropagation against automatic differentiation, and compared them with linear regression on the one-dimensional Runge function and with logistic regression on the MNIST handwritten digits. On Runge's function, small networks approximated the true function about an order of magnitude better than polynomial regression, which oscillated near the ends of the interval, while larger networks overfit unless they were regularized. On MNIST, a network with four hidden layers reached a test accuracy of 97.7%, against 92.6% for logistic regression, mainly by separating digits with similar strokes. The networks were, however, far more expensive to tune and more fragile to train. The lesson is that a network pays off when good features are hard to design by hand, as for images, while a linear model with suitable features remains a cheap and robust choice.**

***Index Terms*—classification, regression, feed-forward neural networks, Runge's function, MNIST**

I. Introduction

In the essay *The Bitter Lesson* [1], Richard Sutton argues that the most significant advances in artificial intelligence have ultimately come from general methods that leverage computation, rather than approaches relying on extensive human-engineered domain knowledge. The step from Project 1 [2] to this project is a small instance of this idea. There, we chose the features ourselves, as powers of x in a polynomial design matrix. A neural network instead takes the raw input, whether a single coordinate or the pixels of an image, and learns its own features from the data.

Hand-made features have clear limits. In Project 1, polynomial fits to Runge's function oscillated near the endpoints of the interval, and logistic regression on raw pixels can only match each image against a fixed template per class, so a digit that is shifted or slanted is easily misclassified. Neural networks remove this limitation through compositions of linear maps and nonlinear activation functions, at the price of a non-convex cost, more hyperparameters and longer training. For a smooth one-dimensional function, however, ordinary least squares is cheap, has a closed-form solution and is hard to beat. The question is therefore not whether a network is more flexible, but when its flexibility is worth its cost.

To answer it, we implement feed-forward neural networks (FFNNs) from scratch, including backpropagation, the cost functions and the ℓ_1 and ℓ_2 penalties, and train them with the gradient descent methods of Project 1. We apply them to two problems: regression of Runge's function, compared with OLS, Ridge and Lasso, and classification of the handwritten digits of the MNIST dataset [3], compared with logistic regression and a majority-class baseline. We study plain and stochastic gradient descent with RMSprop and Adam, the learning rate and its schedule, the depth and width of the network, the activation function, and the penalty, and we compare our implementation with Scikit-Learn [4].

Section II presents the theory and section III the data, the implementation and its verification. Section IV presents and discusses the results and ends with a critical evaluation of the methods, and section V concludes. Appendix A documents our use of large language models, and the code is available in our repository [5].

II. Method

This section presents the cost functions and the gradient-based methods used to minimize them, then the models, linear regression, logistic regression, and neural networks, and finally the evaluation methods used. We assume the reader has familiarity with basic linear algebra and statistics.

A. Cost functions and their gradients

1) Mean squared error: For regression, we measure the fit of the predictions $\tilde{y}_i$ to the targets y_i by the mean squared error,

$$C_{\text{MSE}} = \frac{1}{n}\sum_{i=0}^{n-1}\left(y_i - \tilde{y}_i\right)^2, \tag{1}$$

the same quantity that OLS minimizes. Squaring the residuals penalizes large errors heavily, and up to constants the MSE is the negative log-likelihood of Gaussian noise as in eq. (17) [6]. Its derivative with respect to a prediction is

$$\frac{\partial C_{\text{MSE}}}{\partial \tilde{y}_i} = \frac{2}{n}\left(\tilde{y}_i - y_i\right), \tag{2}$$

which is all that backpropagation needs from the cost. For the linear model $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$, the chain rule recovers

$$\nabla_{\boldsymbol{\theta}} C_{\text{MSE}} = \frac{2}{n}\boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}). \tag{3}$$

2) Binary cross entropy: For binary classification with labels $y_i \in \{0, 1\}$, we replace the Gaussian noise model of regression by a Bernoulli model. The model outputs a logit z_i, and the sigmoid maps it to the probability of the positive class, $p_i = \sigma(z_i) = P(y_i = 1 \mid \boldsymbol{x}_i)$, so that the negative class has probability $1-p_i$. For independent samples, the likelihood of the data given the parameters $\boldsymbol{\theta}$ is

$$P(\mathcal{D} \mid \boldsymbol{\theta}) = \prod_{i=0}^{n-1} p_i^{y_i}(1-p_i)^{1-y_i}, \tag{4}$$

and minimizing its negative logarithm, averaged over the samples, gives the binary cross entropy (BCE) [6], [7],

$$C_{\text{BCE}} = -\frac{1}{n}\sum_{i=0}^{n-1}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right]. \tag{5}$$

A confident, correct prediction contributes almost nothing, while the cost grows without bound as the model becomes confidently wrong. Since $\sigma'(z) = \sigma(z)(1-\sigma(z))$, the derivative with respect to a logit is

$$\frac{\partial C_{\text{BCE}}}{\partial z_i} = \frac{1}{n}\left(p_i - y_i\right). \tag{6}$$

The derivative of the sigmoid has cancelled, so as for the MSE in eq. (2) the gradient is proportional to the residual, and it does not vanish when the sigmoid saturates. For logistic regression, $\boldsymbol{z} = \boldsymbol{X}\boldsymbol{\theta}$, and the chain rule gives

$$\nabla_{\boldsymbol{\theta}} C_{\text{BCE}} = \frac{1}{n}\boldsymbol{X}^\top(\boldsymbol{p} - \boldsymbol{y}), \tag{7}$$

which has the form of eq. (3) with the linear prediction $\boldsymbol{X}\boldsymbol{\theta}$ replaced by the probabilities $\boldsymbol{p}$. Unlike for OLS, setting it to zero has no closed-form solution [7].

3) Categorical cross entropy: For K classes, the model outputs one logit z_{ik} per class, and the softmax maps them to probabilities that are positive and sum to one,

$$p_{ik} = \frac{e^{z_{ik}}}{\sum_{k'=0}^{K-1} e^{z_{ik'}}}. \tag{8}$$

With the targets one-hot encoded, $y_{ik} = 1$ if sample i belongs to class k and 0 otherwise, the negative log-likelihood averaged over the samples is the categorical cross entropy (CCE),

$$C_{\text{CCE}} = -\frac{1}{n}\sum_{i=0}^{n-1}\sum_{k=0}^{K-1} y_{ik}\log p_{ik}, \tag{9}$$

which reduces to eq. (5) for $K = 2$ [7]. Its derivative with respect to a logit has the same form as for the binary case,

$$\frac{\partial C_{\text{CCE}}}{\partial z_{ik}} = \frac{1}{n}\left(p_{ik} - y_{ik}\right), \tag{10}$$

so multinomial logistic regression, with $\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{\Theta}$, has the gradient $\frac{1}{n}\boldsymbol{X}^\top(\boldsymbol{P}-\boldsymbol{Y})$. Since the softmax is unchanged when the same constant is subtracted from all logits, we subtract the largest logit before exponentiating, so that no exponential overflows [7].

4) Penalties: Any of the costs above can be extended with an ℓ_2 or ℓ_1 penalty on the weight matrices $\boldsymbol{W}^l$ of the network,

$$C_{\ell_2} = C + \lambda\sum_l \|\boldsymbol{W}^l\|_2^2, \qquad C_{\ell_1} = C + \lambda\sum_l \|\boldsymbol{W}^l\|_1, \tag{11}$$

with the norms taken element-wise and $\lambda \geq 0$. The penalties add $2\lambda\boldsymbol{W}^l$ and $\lambda\,\text{sign}(\boldsymbol{W}^l)$ to the gradient, respectively. As for Lasso, the ℓ_1 penalty is not differentiable at zero, and we use $\text{sign}(0) = 0$, the minimal-norm subgradient. For the linear model, these are the Ridge and Lasso costs of section II-C.

B. Gradient-based optimization

Apart from OLS and Ridge regression, none of the costs in this report have a closed-form minimizer, so we minimize them iteratively with gradient descent (GD). For a differentiable cost $C(\boldsymbol{\theta})$, GD iterates

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma\nabla_{\boldsymbol{\theta}} C\big(\boldsymbol{\theta}^{(k)}\big), \tag{12}$$

with learning rate $\gamma > 0$, at the cost of one gradient evaluation per iteration [7].

a) Momentum: Momentum adds a fraction $\beta \in [0, 1)$ of the previous step to the current one,

$$\boldsymbol{v}^{(k)} = \beta\boldsymbol{v}^{(k-1)} + \gamma\nabla_{\boldsymbol{\theta}} C\big(\boldsymbol{\theta}^{(k)}\big), \qquad \boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \boldsymbol{v}^{(k)}, \tag{13}$$

with $\boldsymbol{v}^{(0)} = \boldsymbol{0}$. Steps along directions where the gradient keeps its sign accumulate, while oscillations across a narrow valley partly cancel, which helps on ill-conditioned costs [6], [7].

b) Adaptive learning rates: Adaptive methods give each parameter its own effective learning rate. AdaGrad [8] divides the step of each coordinate by the root of its accumulated squared gradients, so the effective step can only decrease. RMSprop replaces the sum by an exponentially decaying average,

$$\begin{aligned}\boldsymbol{v}^{(k)} &= \rho\boldsymbol{v}^{(k-1)} + (1-\rho)\,\boldsymbol{g}^{(k)} \odot \boldsymbol{g}^{(k)},\\ \boldsymbol{\theta}^{(k+1)} &= \boldsymbol{\theta}^{(k)} - \frac{\gamma}{\sqrt{\boldsymbol{v}^{(k)}} + \epsilon} \odot \boldsymbol{g}^{(k)},\end{aligned} \tag{14}$$

so the step size can recover [6]. Here $\boldsymbol{g}^{(k)}$ is the gradient, $\odot$ the Hadamard product, and the square root and division are element-wise. Adam [9] combines a decaying average of the gradient, as in momentum, with the scaling of RMSprop, and corrects both averages for their initialization at zero:

$$\begin{aligned}\boldsymbol{m}^{(k)} &= \beta_1\boldsymbol{m}^{(k-1)} + (1-\beta_1)\,\boldsymbol{g}^{(k)},\\ \boldsymbol{v}^{(k)} &= \beta_2\boldsymbol{v}^{(k-1)} + (1-\beta_2)\,\boldsymbol{g}^{(k)} \odot \boldsymbol{g}^{(k)},\\ \boldsymbol{\theta}^{(k+1)} &= \boldsymbol{\theta}^{(k)} - \gamma\,\frac{\boldsymbol{m}^{(k)}/(1-\beta_1^k)}{\sqrt{\boldsymbol{v}^{(k)}/(1-\beta_2^k)} + \epsilon}.\end{aligned} \tag{15}$$

We use $\rho = 0.9$ for RMSprop, $\beta_1 = 0.9$ and $\beta_2 = 0.999$ for Adam, and $\epsilon = 10^{-8}$. Since Adam divides by the running root-mean-square of the gradient, its steps do not shrink with the gradient near a minimum, and with a fixed γ it keeps taking steps of order γ there.

c) Stochastic gradient descent: For a cost that is an average over the data, $C(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=0}^{n-1} c_i(\boldsymbol{\theta})$, stochastic gradient descent (SGD) replaces the full gradient by the average over a mini-batch $\mathcal{B}_k$ of M points,

$$\boldsymbol{g}^{(k)} = \frac{1}{M}\sum_{i\in\mathcal{B}_k} \nabla_{\boldsymbol{\theta}} c_i\big(\boldsymbol{\theta}^{(k)}\big). \tag{16}$$

This is an unbiased estimate of the full gradient [6]. We shuffle the training set once per epoch and split it into $\lceil n_{\text{tr}}/M \rceil$ mini-batches, so one epoch costs the same number of floating-point operations as one full-batch step but makes $\lceil n_{\text{tr}}/M \rceil$ updates. The gradient noise does not vanish at the minimum, so with a fixed learning rate the iterates fluctuate around it; a decaying learning rate, here $\gamma_t = \gamma_0/(1+\delta t)$ or $\gamma_t = \gamma_0 e^{-\delta t}$ with t the number of updates, damps these fluctuations [7]. Any of the update rules above can be applied to $\boldsymbol{g}^{(k)}$.

C. Linear regression

Consider a dataset of n samples $\mathcal{D} = \{(y_i, x_i)\}_{i=0}^{n-1}$, and assume that the targets y_i are generated by

$$\boldsymbol{y} = \boldsymbol{f} + \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \sigma^2 \boldsymbol{I}_n), \tag{17}$$

where $\boldsymbol{f}$ collects the values of an unknown function f. A polynomial model of degree p can then be written as the prediction $\tilde{\boldsymbol{y}} = \theta_0 + \boldsymbol{X}\boldsymbol{\theta}$, with $\boldsymbol{X} \in \mathbb{R}^{n\times p}$ the design matrix containing the polynomials $\boldsymbol{x}, \ldots, \boldsymbol{x}^p$, and $\boldsymbol{\theta} \in \mathbb{R}^p$ the parameter vector, with θ_0 the intercept. The goal of linear regression is to choose $\hat{\boldsymbol{\theta}}$ so that the fitted model $g(\boldsymbol{X}; \hat{\boldsymbol{\theta}}) = \theta_0 + \boldsymbol{X}\hat{\boldsymbol{\theta}}$ best approximates the unknown function f on the data $\mathcal{D}$ [7], [10].

One solution to this is to minimize the mean squared error (MSE), which yields the ordinary least squares (OLS) predictor,

$$C_{\text{OLS}}(\boldsymbol{\theta}) = \frac{1}{n}\left\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\right\|_2^2. \tag{18}$$

This cost is convex, since its Hessian $\boldsymbol{H} = \frac{2}{n}\boldsymbol{X}^\top\boldsymbol{X}$ is positive semi-definite, so every stationary point is a global minimizer. Setting the gradient $\nabla_{\boldsymbol{\theta}} C_{\text{OLS}} = \frac{2}{n}\boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y})$ to zero gives the normal equations; if $\boldsymbol{X}$ has full column rank, the normal equations have the unique solution

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}. \tag{19}$$

By adding the squared ℓ_2-norm of the parameter vector as a penalty to the cost function, we obtain Ridge regression [11],

$$C_{\text{Ridge}}(\boldsymbol{\theta}) = \frac{1}{n}\left\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\right\|_2^2 + \lambda\|\boldsymbol{\theta}\|_2^2, \qquad \lambda \geq 0. \tag{20}$$

For $\lambda > 0$ the Hessian is positive definite, so the cost is strictly convex. This yields the minimizer

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}}(\lambda) = \left(\boldsymbol{X}^\top\boldsymbol{X} + n\lambda\boldsymbol{I}\right)^{-1}\boldsymbol{X}^\top\boldsymbol{y}. \tag{21}$$

Lasso [12] instead applies an ℓ_1 penalty,

$$C_{\text{Lasso}}(\boldsymbol{\theta}) = \frac{1}{n}\left\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\right\|_2^2 + \lambda\|\boldsymbol{\theta}\|_1. \tag{22}$$

The cost is convex but not differentiable where a coefficient is zero, and it has no closed-form minimizer [10], so it is minimized with the methods of section II-B. Lasso performs variable selection while Ridge only shrinks [7].

D. Logistic regression

Logistic regression is a natural choice as a baseline. It is a simple model in which everything can be computed, its cost function is convex, and it is exactly a neural network with no hidden layers [7]. Its cost is the binary or categorical cross entropy of section II-A.

E. Neural networks

Linear and logistic regression share a common limitation: both act on a fixed design matrix $\boldsymbol{X}$, so the model can only be as expressive as the features we choose to put into it. A feed-forward neural network removes this limitation by inserting one or more *hidden layers* between the input and the output, each applying a linear map followed by a non-linear activation. The final layer is then nothing more than a linear or logistic regression, but it acts on features that are *learned* from the data rather than given in advance [7].

1) The artificial neuron: An artificial neuron computes a linear predictor, as in logistic regression, followed by an activation function. We number layers as $l = 0, 1, \ldots, L$, with $l = 0$ the input and $l = L$ the output, and n_l the number of nodes in layer l. For a single neuron j in layer l, the activation is

$$z_j^l = \sum_{i=0}^{n_{l-1}-1} w_{ij}^l a_i^{l-1} + b_j^l, \tag{23}$$

where w_{ij}^l is the weight connecting node i of layer $l-1$ to node j of layer l, and b_j^l is the bias of node j in layer l. We can then obtain the output by applying the activation function [7],

$$a_j^l = f(z_j^l). \tag{24}$$

This whole process is depicted in fig. 1, where the activation and output of a single neuron j in layer l is shown.

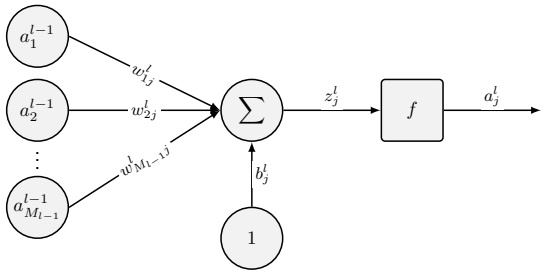


Fig. 1. The activation and output of a single neuron j in layer l, as shown in eqs. (23) and (24).

2) Forward pass: We can write the activation and output of all neurons in the layer l in matrix-form, where eqs. (23) and (24) become

$$\boldsymbol{z}^l = (\boldsymbol{W}^l)^\top\boldsymbol{a}^{l-1} + \boldsymbol{b}^l, \qquad \boldsymbol{a}^l = \boldsymbol{f}(\boldsymbol{z}^l), \tag{25}$$

with $\boldsymbol{f}$ applied element-wise. Evaluating eq. (25) sequentially for all l-values is the feed-forward pass and returns the network output $\tilde{\boldsymbol{y}} = \boldsymbol{a}^L$ [7].

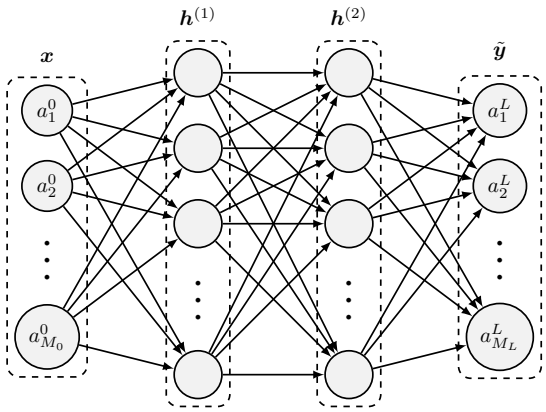


Fig. 2. Illustration of a fully connected feed-forward neural network. Adapted from Love [13].

We can further collect the weights and biases into a single parameter vector $\boldsymbol{\Theta} = \{\boldsymbol{W}^l, \boldsymbol{b}^l\}_{l=1}^L$, and write the optimization problem as

$$\hat{\boldsymbol{\Theta}} = \arg\min_{\boldsymbol{\Theta}} C(\boldsymbol{\Theta}), \quad C(\boldsymbol{\Theta}) = \frac{1}{n}\sum_{i=0}^{n-1} c_i(\boldsymbol{\Theta}), \tag{26}$$

where c_i is the cost of sample i for one of the cost functions of section II-A, with a penalty from eq. (11) added when the network is regularized [7]. This is a non-convex optimization problem, so there is no closed-form solution, and we must resort to the gradient-based methods reported in section II-B to find a local minimum.

3) Activation functions: The activation function must be non-linear and differentiable almost everywhere. If it were linear, the composition of multiple layers would collapse into a single affine transformation, and the network would be no more expressive than a single layer [7]. Our study will revolve around the following three activation functions:

a) Sigmoid: The logistic or sigmoid function is given by

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)(1-\sigma(z)),$$

and was the standard choice for decades. Its main drawback is that it saturates for large $|z|$, which leads to vanishing gradients and slow training [7].

b) ReLU: The rectified linear unit (ReLU),

$$\mathrm{ReLU}(z) = \max(0, z), \quad \mathrm{ReLU}'(z) = \begin{cases} 1, & \text{if } z > 0, \\ 0, & \text{if } z \leq 0, \end{cases}$$

is faster to compute, does not saturate for $z > 0$, and is a sound default choice for hidden layers. Its main drawback is that it can lead to dead neurons, which never activate and thus never learn [7].

c) Leaky ReLU: The leaky ReLU repairs this issue by allowing a small gradient for negative values, so the gradient never completely vanishes. It is defined as

$$\text{Leaky ReLU}(z) = \begin{cases} z, & \text{if } z > 0, \\ \alpha z, & \text{if } z \leq 0, \end{cases} \quad \alpha \approx 0.01,$$

and its derivative is

$$\text{Leaky ReLU}'(z) = \begin{cases} 1, & \text{if } z > 0, \\ \alpha, & \text{if } z \leq 0. \end{cases}$$

4) Backpropagation: We can compute the gradient of the cost function with respect to the parameters using the backpropagation algorithm [14]. The key idea is to compute the error at the output layer and then propagate it backward through the network.

Writing the cost as $C = \frac{1}{n}\sum_{i=0}^{n-1} c_i$, the error of sample i at layer l is $\boldsymbol{\delta}_i^l = \partial c_i / \partial \boldsymbol{z}_i^l$, which satisfies

$$\boldsymbol{\delta}_i^l = (\boldsymbol{W}^{l+1}\boldsymbol{\delta}_i^{l+1}) \odot \boldsymbol{f}'(\boldsymbol{z}_i^l),$$

where $\odot$ denotes the element-wise product. At the output, all three pairings of output layer and cost give $\boldsymbol{\delta}_i^L \propto \boldsymbol{a}_i^L - \boldsymbol{y}_i$, with a factor 2 for the MSE and 1 for the cross entropies [7]. The gradients are then averaged over the samples,

$$\frac{\partial C}{\partial \boldsymbol{W}^l} = \frac{1}{n}\sum_{i=0}^{n-1} \boldsymbol{a}_i^{l-1}(\boldsymbol{\delta}_i^l)^\top, \qquad \frac{\partial C}{\partial \boldsymbol{b}^l} = \frac{1}{n}\sum_{i=0}^{n-1} \boldsymbol{\delta}_i^l,$$

where SGD takes the mean over a minibatch instead. Any penalty gradient (eq. (11)) is added to $\partial C/\partial \boldsymbol{W}^l$, and the parameters are updated with the methods of section II-B.

5) Choice of output layer and loss function: For regression, an identity output allows unrestricted continuous predictions and is paired with the MSE. For binary classification, a sigmoid output gives a probability in $[0, 1]$ and is paired with the binary cross-entropy, and for multiclass classification, the softmax gives a probability distribution over the classes and is paired with the categorical cross-entropy. These pairings follow from the assumed distribution of the target and the corresponding likelihood [7], [15].

6) Weight initialization: Unlike the linear models, the network has no closed-form solution, and gradient descent must start from some initial weights. Setting all weights to zero would give every node in a layer the same output and the same gradient, so they would stay identical; the initialization must break this symmetry. Weights that are too large saturate the activations, while weights that are too small make the signal vanish [7]. Glorot and Bengio [16] argue that the variance of the activations should be kept from layer to layer in the forward pass, and the variance of the gradients in the backward pass. For a layer with n_{l-1} inputs and n_l nodes, this gives Xavier initialization, while He et al. [17] double the variance to make up for ReLU setting half of its inputs to zero:

$$\begin{aligned} W_{ij}^l &\sim \mathcal{N}\left(0, \frac{2}{n_{l-1}+n_l}\right) \quad \text{(Xavier)}, \\ W_{ij}^l &\sim \mathcal{N}\left(0, \frac{2}{n_{l-1}}\right) \quad \text{(He)}. \end{aligned} \tag{27}$$

We use Xavier for the hidden layers of sigmoid networks and He for those of the ReLU family, and initialize all biases to zero.

7) Regularization: A network with more parameters than data points available can fit the entire training set exactly, so applying regularization is essential [7]. We add the penalties of eq. (11), which play the same role for the network as Ridge and Lasso do for the linear model. With plain gradient descent, the ℓ_2 penalty turns the weight update into

$$\boldsymbol{W}^l \leftarrow (1 - 2\gamma\lambda)\, \boldsymbol{W}^l - \gamma \frac{\partial C}{\partial \boldsymbol{W}^l}, \tag{28}$$

so each step shrinks all weights towards zero by the same factor, which is why it is also known as weight decay [6]. The ℓ_1 penalty instead reduces the magnitude of each weight by the constant $\gamma\lambda$, which favors sparse weights, although with a subgradient and a finite step the small weights oscillate around zero rather than becoming exactly zero. With RMSprop and Adam, the penalty gradient is rescaled together with the rest of the gradient, so the shrinkage is no longer the same for all weights. The biases are not penalized: each bias only shifts the input to a single node, so it adds little flexibility to the network, and penalizing it can cause underfitting [6]. The strength λ is a hyperparameter, chosen on validation data together with the others.

Early stopping is a form of regularization that limits overfitting by terminating training when performance on a validation set ceases to improve [7]. While the training loss may continue to decrease, an increasing validation loss can indicate that the model is beginning to overfit the training data [18]. We therefore monitor the validation loss throughout training and stop if it has not decreased for a given number of epochs, the patience. The model weights are subsequently restored to those obtained at the epoch with the lowest validation loss.

F. Evaluation

A model cannot be trusted without any metrics for evaluating its performance. For regression, we evaluate a prediction $\tilde{\boldsymbol{y}}$ of targets $\boldsymbol{y}$ by

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=0}^{n-1}(y_i - \tilde{y}_i)^2, \quad R^2 = 1 - \frac{\sum_{i=0}^{n-1}(y_i - \tilde{y}_i)^2}{\sum_{i=0}^{n-1}(y_i - \bar{y})^2}, \tag{29}$$

with $\bar{y}$ the mean of $\boldsymbol{y}$. An unbiased model with no variance reaches $\mathrm{MSE} = \sigma^2$ in expectation; R^2 compares against predicting $\bar{y}$ and is negative for a model that does worse than that constant [7].

For classification, the predicted class $\tilde{y}_i = \arg\max_k p_{ik}$ is the class with the highest predicted probability, and the simplest measure is the accuracy,

$$\mathrm{Accuracy} = \frac{1}{n}\sum_{i=0}^{n-1} I(\tilde{y}_i = y_i), \tag{30}$$

where the indicator function I is 1 if the prediction is correct and 0 otherwise [7]. A single number hides which classes are confused with which, so we also compute the confusion matrix $\boldsymbol{N}$, where N_{jk} counts the samples of true class j predicted as class k. Correct predictions lie on the diagonal, and the accuracy is $\mathrm{tr}(\boldsymbol{N})/n$. From $\boldsymbol{N}$, the precision and recall of class k are

$$\mathrm{Precision}_k = \frac{N_{kk}}{\sum_j N_{jk}}, \qquad \mathrm{Recall}_k = \frac{N_{kk}}{\sum_j N_{kj}}, \tag{31}$$

the fraction of predictions of class k that are correct, and the fraction of samples of class k that are found. Normalizing each row of $\boldsymbol{N}$ by its sum therefore puts the recall on the diagonal. The F_1 score of class k is the harmonic mean of the two,

$$F_{1,k} = 2\,\frac{\mathrm{Precision}_k \cdot \mathrm{Recall}_k}{\mathrm{Precision}_k + \mathrm{Recall}_k}, \tag{32}$$

which is high only if both are. We summarize the per-class scores by their unweighted mean, the macro average, which weights every class equally regardless of its size [7].

III. Implementation

All methods were implemented from scratch in Python using NumPy [19], reusing the data generation, linear models and optimizers of Project 1 [2]. Scikit-Learn [4] is used only for splitting and scaling the data and as a reference implementation. The code and the notebooks reproducing every figure and table are available in the project repository [5].

A. Data and preprocessing

For the regression task, we use Runge's function [20], a classical example for studying the bias–variance trade-off. It is defined as

$$f(x) = \frac{1}{1 + 25x^2}, \qquad x \in [-1, 1]. \tag{33}$$

As observed in Project 1 [2], polynomial regression can exhibit substantial oscillations near the endpoints of this interval. We generate $n = 100$ input samples uniformly from $[-1, 1]$, evaluate the function at these inputs, and add i.i.d. Gaussian noise with standard deviation $\sigma = 0.1$ to obtain the observed targets. As in Project 1, 20% of the samples are held out for testing, with the same seed, so that the networks and the linear models are compared on identical data. The networks take the raw x as their only input feature, standardized with the mean and standard deviation of the training data, while the linear models use standardized polynomial features.

For the classification task, we use the MNIST dataset [3], which contains 70 000 grayscale images of handwritten digits, each of size 28×28 pixels, with labels corresponding to the ten classes $0, \ldots, 9$, split into 60 000 training and 10 000 test images. We flatten each image to a vector of 784 features scaled to $[0, 1]$, encode the labels as one-hot vectors, and set aside a stratified validation set of 10 000 training images. All hyperparameters are selected on the validation set; the test set is used only once, for the final comparison of the selected models.

B. Code structure

Dense layers and activation functions are separate layers of the network, so any activation can be placed after any layer, and the weights are initialized as in eq. (27). The sigmoid and the softmax are part of the cross entropies, so classification networks end in a linear layer that outputs logits. Logistic regression is the same network with a single dense layer. The network is trained as in algorithm 1.

Algorithm 1 Training of the FFNN with minibatches, learning rate scheduling and early stopping

Require: $\boldsymbol{\Theta}^{(0)}$, schedule γ_t, batch size M, maximum epochs E, patience P, update rule $\mathcal{U}$ with state $\boldsymbol{s}$

$t \leftarrow 0$, $\boldsymbol{s} \leftarrow \mathbf{0}$, $C^* \leftarrow \infty$, $e^* \leftarrow 0$
for $e = 1, \dots, E$ **do**
  Shuffle the training rows; split into $\lceil n_{\text{tr}}/M \rceil$ batches
  for each batch $\mathcal{B}$ **do**
    $\boldsymbol{g} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\Theta}} c_i(\boldsymbol{\Theta}^{(t)})$ ▷ backpropagation
    $(\boldsymbol{\Theta}^{(t+1)}, \boldsymbol{s}) \leftarrow \mathcal{U}(\boldsymbol{\Theta}^{(t)}, \boldsymbol{g}, \gamma_t, \boldsymbol{s})$ ▷ e.g. eqs. (12) and (15)
    $t \leftarrow t + 1$
  end for
  $C_{\text{val}} \leftarrow C(\boldsymbol{\Theta}^{(t)})$ on the validation data
  if $C_{\text{val}} < C^*$ **then**
    $C^* \leftarrow C_{\text{val}}$, $e^* \leftarrow e$, $\boldsymbol{\Theta}^* \leftarrow \boldsymbol{\Theta}^{(t)}$
  else if $e - e^* \geq P$ **then**
    break
  end if
end for
return $\boldsymbol{\Theta}^*$

C. Hyperparameter search

All searches are grid searches, and every configuration starts from the same initial weights. The scores are computed from the predictions, so they never include the penalty, and networks with different λ are compared on equal terms. For regression, configurations are scored by their 5-fold cross-validated MSE on the training data; the grids are given with the corresponding figures.

For classification, configurations are scored by their accuracy on the validation set. All models are trained with Adam and a batch size of 128, logistic regression for 50 epochs over a grid of γ and λ, which selects $\gamma = 10^{-3}$ and $\lambda = 10^{-6}$. The networks are trained for at most 50 epochs with early stopping with a patience of 5 epochs, and an exponentially decaying learning rate that falls by a factor of 10 over 50 epochs. We first search over the activation, the depth, the width and γ without regularization, and then over γ and λ for the best architecture (table IV). The selected network is retrained with the same procedure, so the reported model is exactly the one that was scored.

D. Verification

The code is covered by unit tests. Most importantly, the gradients from backpropagation agree with those from automatic differentiation with JAX [21] to a relative tolerance of 10^{-7}, for networks with two hidden layers and every combination of activation function, cost function and penalty. We also compare our logistic regression and our selected network with Scikit-Learn's `LogisticRegression` and `MLPClassifier`. These normalize the penalty differently: our λ corresponds to $C = 1/(2\lambda n)$ and $\alpha = 2\lambda M$, respectively.

IV. Results and Discussion

We first compare the networks with the linear models on Runge's function, and then with logistic regression on MNIST. The section ends with a critical evaluation of the methods.

A. Regression

The test set has only 20 samples, and the noise variance $\sigma^2 = 0.01$ sets a floor for the test MSE that every good model reaches. Differences in test MSE between good models are therefore mostly noise, so we compare them by their MSE against the true function f on a dense grid, which is free of noise.

1) Sigmoid networks and optimizers: Figure 3 shows the main result of this part. OLS with the polynomial degree selected in Project 1 oscillates near the ends of the interval, while both networks follow Runge's function smoothly, using only the raw x as input.

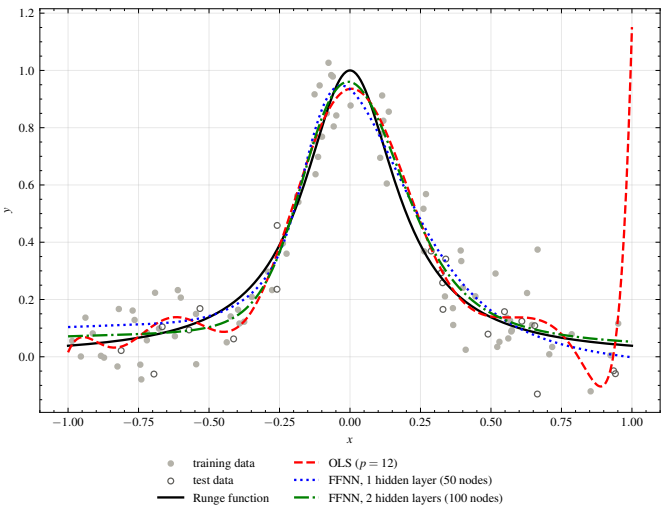


Fig. 3. OLS and two sigmoid networks fitted to 80 noisy samples of Runge's function. OLS oscillates near the ends of the interval, while the networks follow the function smoothly.

OLS has the lowest training MSE, but the networks approximate f about an order of magnitude better, as shown in table I. The polynomial spends its flexibility on fitting the noise near the edges, where the data constrain it least.

TABLE I
PERFORMANCE OF OLS AND THE TWO SIGMOID NETWORKS. THE NETWORKS HAVE A HIGHER TRAINING MSE, BUT ARE FAR CLOSER TO THE TRUE FUNCTION. THE 1×50 NETWORK IS TRAINED WITH ADAM ($\gamma = 10^{-1}$, 100 EPOCHS) AND THE 2×100 NETWORK WITH RMSPROP ($\gamma = 10^{-4}$, 10^4 EPOCHS), BOTH WITH A BATCH SIZE OF 32.

Model	Train MSE	Test MSE	MSE vs f
OLS ($p = 12$)	0.0081	0.0086	0.0125
FFNN (1×50)	0.0097	0.0092	0.0020
FFNN (2×100)	0.0089	0.0085	0.0012

Figure 4 shows how the optimizer and the learning rate decide whether the networks train at all. Plain gradient descent rarely leaves the mean prediction, while RMSprop and Adam reach the noise floor over a wide range of learning rates. Minibatches make every optimizer more robust, most clearly at large learning rates.

Training a sigmoid network often starts with a plateau, where the network predicts the mean of the data and the gradient towards curved features is small [22] (fig. 13). Figure 5

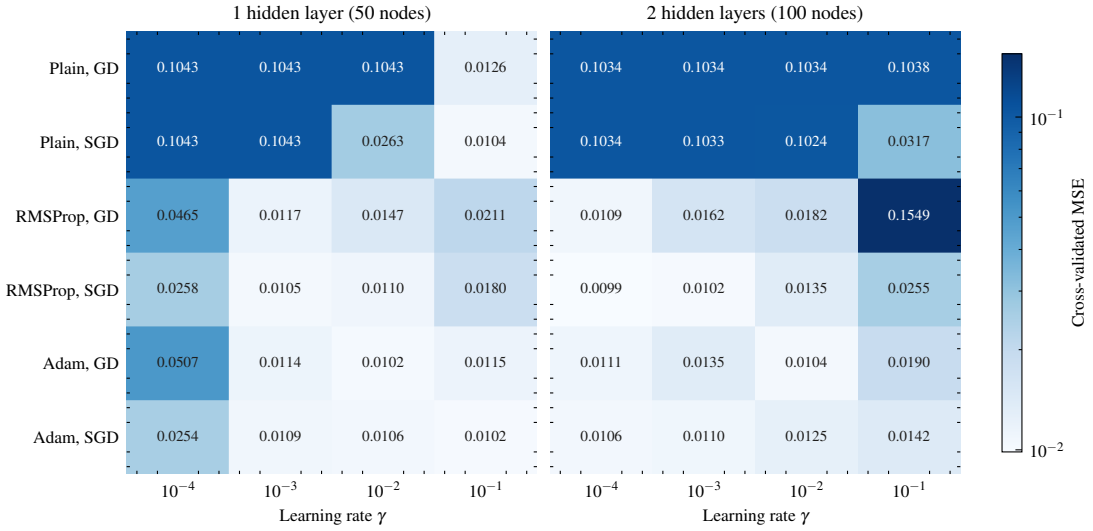


Fig. 4. Plain gradient descent rarely leaves the mean prediction, while the adaptive methods work over a wide range of learning rates. Cross-validated MSE of the two sigmoid networks for each optimizer, with full-batch (GD) and mini-batch (SGD) updates, at the best number of epochs and batch size. An MSE near 0.10 is the mean prediction.

shows that the optimizer decides how long this lasts: RMSprop and Adam leave the plateau far sooner than plain SGD at the same learning rate. A larger learning rate also leaves it sooner, but makes the cost noisy near the minimum, which motivates the decaying learning rate we use for classification.

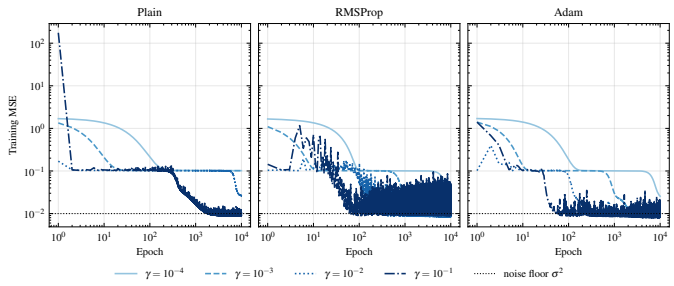


Fig. 5. Training MSE per epoch of the 1×50 sigmoid network, trained with plain SGD, RMSprop and Adam at batch size 16 for four learning rates. The adaptive methods leave the initial plateau far sooner, and larger learning rates are faster but noisier.

2) Activation functions, depth and width: Figure 6 shows that small networks are best for all three activations, and that the error grows with depth, faster for the ReLU family than for the sigmoid. A smooth function of one variable needs little capacity, and the extra parameters of a large network are spent on the noise.

The best network of each activation reaches about the same test MSE (table II), well within the noise of the small test set. Only the MSE against f separates them, in favor of the sigmoid. The ReLU networks are piecewise linear and approximate the peak with a few straight segments, while the sigmoid gives a smooth curve that suits Runge's function better (fig. 14).

The learning curves in fig. 7 confirm that the large networks overfit: their training MSE falls below the noise floor, while their validation MSE stays well above it. The smallest

TABLE II
PERFORMANCE OF THE NETWORK WITH THE LOWEST CROSS-VALIDATED MSE FOR EACH ACTIVATION. ONLY THE MSE AGAINST f SEPARATES THEM.

Activation	Train MSE	Test MSE	MSE vs f
Sigmoid (1×80)	0.0091	0.0076	0.0015
ReLU (2×10)	0.0083	0.0080	0.0022
Leaky ReLU (2×10)	0.0083	0.0080	0.0021

networks show a much smaller gap.

3) Regularization: To counter this overfitting, we add the ℓ_1 and ℓ_2 penalties to the networks with four and eight hidden layers of 40 and 80 nodes. Figure 8 shows the result, and fig. 18 shows the full search over λ and γ.

For the ReLU family, both penalties give the U-shaped curve of the bias–variance trade-off. Without a penalty, the networks fit the noise. A moderate penalty raises the training MSE towards σ^2 and lowers the validation MSE, while a large one shrinks the weights until the network predicts the mean. The penalty helps most for the networks that overfit most, and brings them to the level of the best small networks without a penalty, so for the ReLU family, a penalty can replace a careful choice of architecture. The two norms perform almost identically, and since we use a subgradient with a finite step, ℓ_1 does not give the exactly sparse weights of Lasso either.

The sigmoid networks gain nothing from a penalty: their best configuration has $\lambda = 0$, and even small penalties make them predict the mean. A possible explanation is how the weights scale. A ReLU network can shrink the weights of one layer and grow those of another without changing its output, and can therefore reduce the penalty while keeping its function. A sigmoid network cannot, since small weights put every node in the nearly linear region around $z = 0$, where the network loses the curvature it needs for the peak. The sigmoid networks also overfit less to begin with, so they have less to

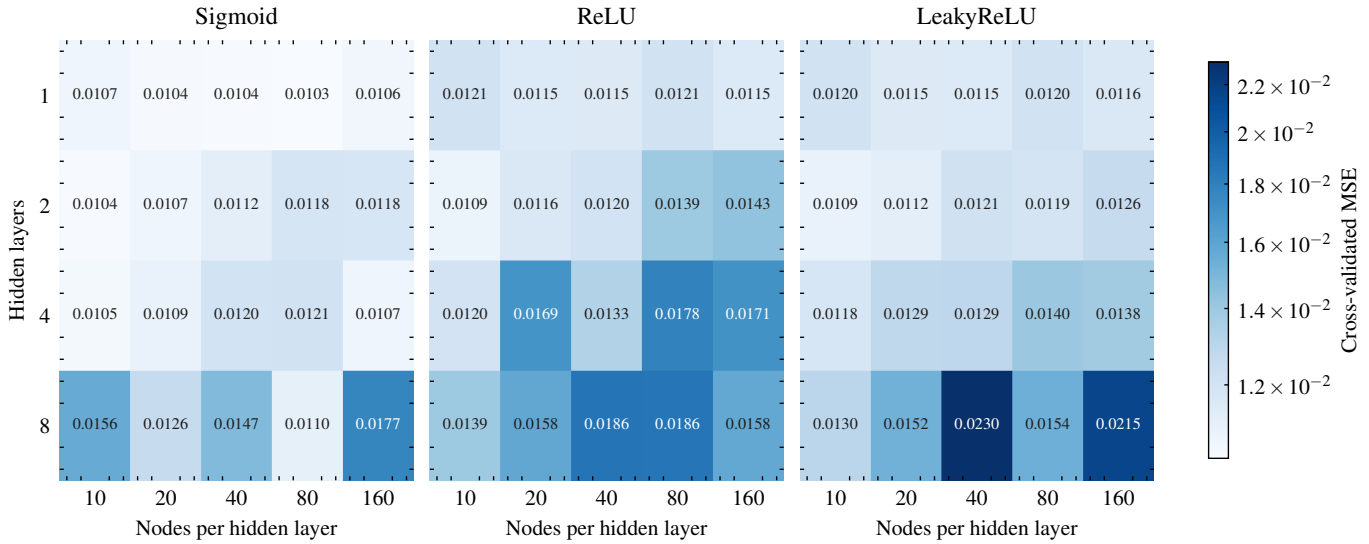


Fig. 6. Cross-validated MSE for each activation and architecture, at the best learning rate $\gamma \in \{10^{-2}, 10^{-3}, 10^{-4}\}$, with Adam, 5000 epochs and a batch size of 32. Lighter is better. Shallow networks are best, and depth hurts the ReLU family most.

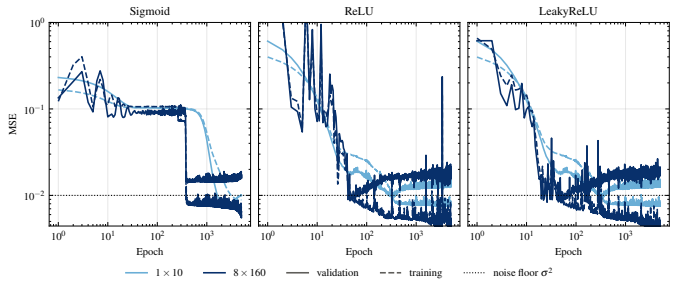


Fig. 7. Training (dashed) and validation (solid) MSE per epoch for the smallest (1×10) and largest (8×160) network of each activation, validated on one fifth of the training data. The large networks fit the noise, with a training MSE below σ^2 and a validation MSE well above it.

gain.

Finally, the best regularized networks approximate f about five times better than the Ridge and Lasso models (table III). Ridge still oscillates near $x = 1$, while Lasso underfits the peak (fig. 15). The Lasso model is weaker than it could be, since our Lasso solver did not converge for high degrees.

TABLE III
THE BEST REGULARIZED NETWORKS AND THE RIDGE AND LASSO MODELS OF PROJECT 1. THE NETWORKS ARE ABOUT FIVE TIMES CLOSER TO THE TRUE FUNCTION.

Model	Train MSE	Test MSE	MSE vs f
Ridge ($p = 11$, $\lambda = 10^{-6}$)	0.0084	0.0100	0.0097
ReLU (4×40), ℓ_2, $\lambda = 10^{-3}$	0.0083	0.0076	0.0016
Lasso ($p = 6$, $\lambda = 10^{-2.5}$)	0.0202	0.0187	0.0101
ReLU (4×80), ℓ_1, $\lambda = 10^{-3}$	0.0086	0.0085	0.0022

B. Image classification

1) Logistic regression: Logistic regression is the network with no hidden layers, so each class score is a weighted sum of the raw pixels. It reaches a test accuracy of about 93% (table V), far above the 11% of always guessing the most common digit.

The errors are not spread evenly. Digits with a distinctive shape, such as 0 and 1, are almost always recognized, while digits that share strokes, such as 3, 5 and 8, are often confused (fig. 16). The weights explain why: each digit has a single template tied to fixed pixel positions (fig. 9), so a digit that is shifted, slanted or written in an unusual style matches the wrong template. Handling this variation requires features that are learned from the data, which is what we expect the hidden layers to provide.

2) Neural networks: Table IV summarizes the search and the selected network. Figure 10 shows that width matters more than depth: for all activations, going from 20 to 80 nodes per layer helps, while more than four hidden layers gives nothing for the ReLU family. The sigmoid networks behave differently. They need more epochs, prefer large learning rates, and fail when the network is deep, which is consistent with the vanishing gradients discussed in section II-E. The ReLU family trains well over a wide range of learning rates, but fails at $\gamma = 10^{-1}$. The best network is both the widest and the shallowest in the grid, so a wider or shallower network might have done even better.

TABLE IV
SEARCH SPACE AND SELECTED CONFIGURATION OF THE CLASSIFICATION NETWORK, CHOSEN BY THE VALIDATION ACCURACY.

Hyperparameter	Search space	Selected
Activation function	Sigmoid, ReLU, Leaky ReLU	ReLU
Hidden layers	$\{4, 8, 12\}$	4
Nodes per layer	$\{20, 40, 80\}$	80
Learning rate γ	$\{10^{-5}, \ldots, 10^{-1}\}$	10^{-2}
ℓ_2 penalty λ	$\{0, 10^{-4}, \ldots, 10^{-1}\}$	10^{-4}

For the selected architecture, a small penalty gives a slight improvement, while $\lambda > 10^{-2}$ clearly hurts (fig. 17). With

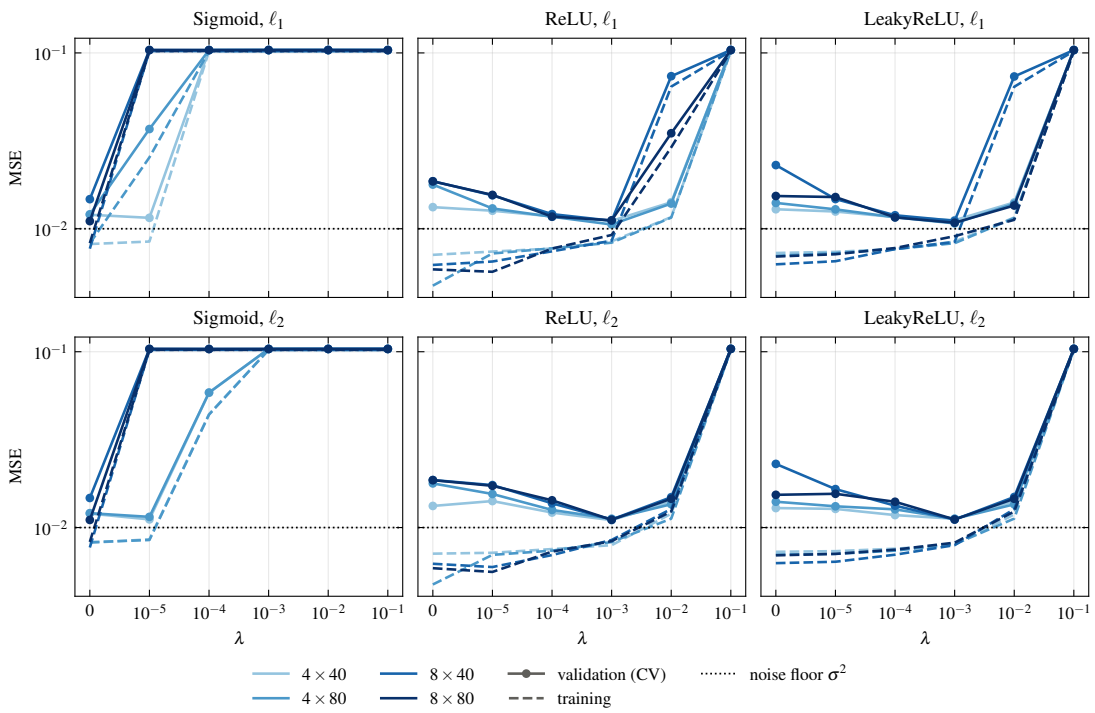


Fig. 8. Training (dashed) and cross-validated (solid) MSE as functions of the penalty λ, at the best learning rate, with $\lambda = 0$ drawn one decade below the smallest λ. For the ReLU family, both penalties give a clear minimum at $\lambda = 10^{-3}$, while every sigmoid network collapses to the mean prediction already at small λ.

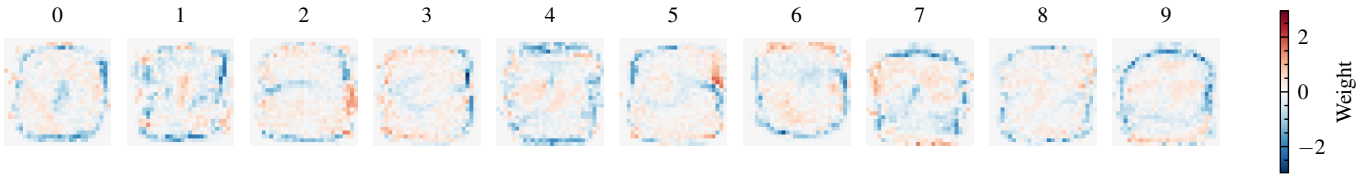


Fig. 9. Weights of logistic regression for each digit, reshaped to 28×28. Red pixels increase the score of the digit and blue pixels decrease it. Each digit is recognized by a single, blurred template at a fixed position.

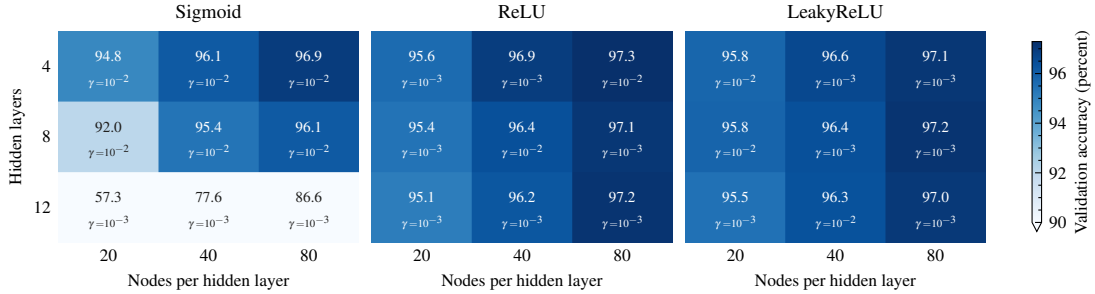


Fig. 10. Validation accuracy of each architecture without a penalty, at its best learning rate (shown in each cell). Wider networks are better for all activations, while depth beyond four layers does not help the ReLU family and harms the sigmoid.

50 000 training images and early stopping, the network does not overfit much, so there is little for the penalty to correct. Large penalties combined with large learning rates make training fail, and the network falls back to predicting a single digit.

Figure 11 shows what the learning rate schedule and early stopping add. With a constant learning rate, the validation cost stops improving early and becomes noisy, while the decaying learning rate keeps lowering it and gives a clearly better network. Early stopping ends training at about half the maximum number of epochs, with almost the same accuracy as training for all of them.

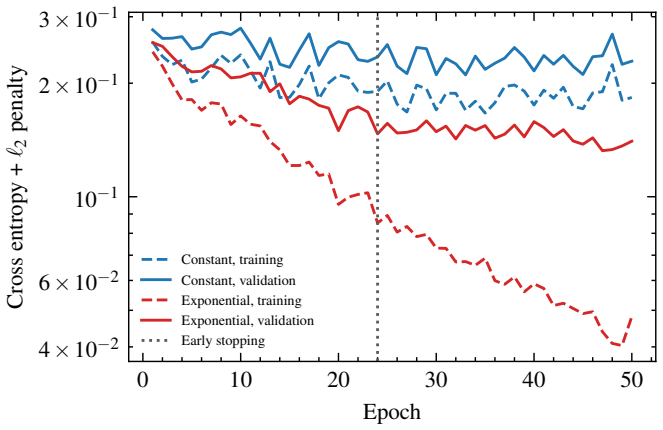


Fig. 11. Training (dashed) and validation (solid) cost per epoch of the selected network, trained for all 50 epochs with a constant and an exponentially decaying learning rate. The decay gives a lower validation cost, and early stopping (dotted) ends training where the gains have become small.

3) Comparison on the test set: On the test set, the network reduces the error rate of logistic regression by a factor of three (table V). The improvement is largest for the digits that logistic regression confused most, such as 3, 5 and 8 (fig. 12). The accuracy and the macro-averaged F_1 score are almost equal for all models, since MNIST is close to balanced.

Our implementations agree well with Scikit-Learn. Our logistic regression is slightly better, and our network is about 1.5 percentage points better than `MLPClassifier` with the same architecture, which has neither our learning rate schedule nor early stopping. This gap is similar to the one between the constant and decaying learning rate in fig. 11.

TABLE V
TEST ACCURACY AND MACRO-AVERAGED F_1 SCORE OF THE SELECTED MODELS.

Model	Accuracy	Macro F_1
Majority class	0.114	0.020
Logistic regression	0.926	0.925
Logistic regression (Scikit-Learn)	0.922	0.921
FFNN	0.977	0.977
FFNN (Scikit-Learn)	0.962	0.961

C. Critical evaluation

For both tasks, the neural networks gave the best models, but at a much higher cost. Linear and logistic regression have few hyperparameters, train quickly and have interpretable weights; OLS and Ridge even have closed-form solutions, and the cost of logistic regression is convex. A network has a non-convex cost, needs a search over the architecture, the activation, the optimizer, the learning rate and the penalty, and each configuration must be trained for many epochs. The networks were also more fragile: too large a learning rate or too deep a sigmoid network made training fail, which never happened for the linear models.

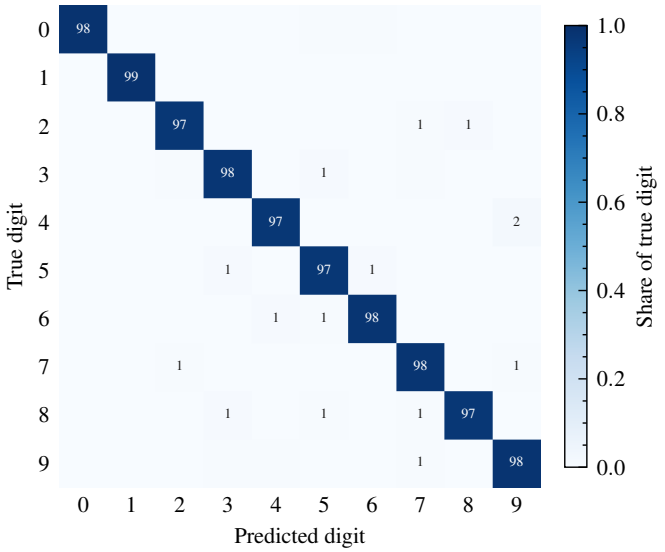


Fig. 12. Confusion matrix of the selected network on the test set, normalized by row (%). Compared with logistic regression (fig. 16), the recall is high for every digit.

For regression, a small sigmoid network approximated Runge's function best. Its advantage over OLS and Ridge comes from the polynomial features, which oscillate near the ends of the interval, rather than from linear regression itself; a better choice of basis would likely narrow the gap. If cost or interpretability matters, Ridge is a reasonable choice, but for the best fit of a function with a sharp feature, the network is better.

For classification, the network made about a third as many errors as logistic regression. Here the features are hard to design by hand, and learning them is what makes the difference. Logistic regression remains a strong and cheap baseline, and its weights show directly what the model has learned.

Across both tasks, Adam and RMSprop were more robust than plain gradient descent, and ReLU networks trained more reliably than sigmoid networks when the network was deep. A penalty only helped where the networks overfit, which was the case for the large networks on the 80 regression samples, but hardly on the 50 000 MNIST images. A decaying learning rate and early stopping improved the classification network at no extra cost.

Our study also has limitations. Every configuration was trained from one set of initial weights and evaluated on one split, so small differences between configurations are not significant. The regression test set has only 20 samples, and the grids are coarse. The best classification network lies at the

edge of the search grid, which starts at four hidden layers, so we cannot say whether one or two hidden layers would have done as well. The learning rate schedule was only used for classification. The answer to the question in section I is therefore qualitative: the flexibility of a network is worth its cost when good features are hard to choose by hand, as for images, and least worth it when a simple model with suitable features is available.

V. Conclusion

We implemented feed-forward neural networks from scratch, verified the gradients from backpropagation against automatic differentiation, and compared the networks with linear and logistic regression on two problems.

On Runge's function, small networks approximated the true function about an order of magnitude better than the polynomial models of Project 1, which oscillate near the ends of the interval. Small networks were best, since larger ones fit the noise; for ReLU networks, an ℓ_1 or ℓ_2 penalty removed most of this overfitting, while sigmoid networks did not benefit from it. On MNIST, a network with four hidden ReLU layers reached a test accuracy of 97.7%, against 92.6% for logistic regression, and corrected most of the confusion between digits that share strokes. Here width mattered more than depth, and a decaying learning rate and early stopping gave a clear improvement. Our implementation performed on par with or better than Scikit-Learn for both models.

The networks were the most accurate models for both tasks, but also the most expensive and the most sensitive to their hyperparameters. Their flexibility paid off most for images, where good features are hard to design by hand. As discussed in section IV-C, the conclusions rest on single training runs, a small regression test set and coarse search grids.

Natural extensions are to repeat the experiments over several random initializations to measure the variation between runs, to search wider networks for MNIST, and to use the learning rate schedule also for regression. Dropout and batch normalization could reduce overfitting and ease the training of deep sigmoid networks, and convolutional networks would exploit the spatial structure of the images that a fully connected network ignores. Finally, the methods could be tested on harder data, such as Runge's function in two or three dimensions and the Fashion-MNIST data set.

The use of large language models in this project is declared in Appendix A.

References

[1] R. S. Sutton, *The bitter lesson*, Mar. 2019. [Online]. Available: http://www.incompleteideas.net/IncIdeas/BitterLesson.html

[2] S. Riseth, *FYS-STK4155 Project 1*, version v0.1.0, GitHub, Oct. 2026. [Online]. Available: https://github.com/sigurdriseth/FYS-STK4155-P1

[3] Y. LeCun, C. Cortes, and C. J. Burges, "MNIST handwritten digit database," *ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist*, vol. 2, 2010.

[4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011. [Online]. Available: http://jmlr.org/papers/v12/pedregosa11a.html

[5] S. Riseth, *FYS-STK4155 Project 2*, version v0.1.0, GitHub, Nov. 2026. [Online]. Available: https://github.com/sigurdriseth/FYS-STK4155-P2

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. Accessed: Aug. 12, 2026. [Online]. Available: https://www.deeplearningbook.org/

[7] M. Hjorth-Jensen, *Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning*. Springer, Oct. 9, 2026. [Online]. Available: https://github.com/EducationalMaterialUiO / MachineLearningUiO / blob / main/doc/BookML/Lecturebook.pdf

[8] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *Journal of Machine Learning Research*, vol. 12, no. 61, pp. 2121–2159, 2011. [Online]. Available: http://jmlr.org/papers/v12/duchi11a.html

[9] D. P. Kingma and J. Ba. "Adam: A Method for Stochastic Optimization." arXiv: 1412.6980 [cs.LG], Accessed: Sep. 21, 2026. [Online]. Available: http://arxiv.org/abs/1412.6980 pre-published.

[10] P. Mehta et al., "A high-bias, low-variance introduction to Machine Learning for physicists," *Physics Reports*, vol. 810, pp. 1–124, May 2019, ISSN: 03701573. DOI: 10.1016/j.physrep.2019.03.001 Accessed: Aug. 26, 2026. [Online]. Available: http://arxiv.org/abs/1803.08823

[11] A. E. Hoerl and R. W. Kennard, "Ridge Regression: Biased Estimation for Nonorthogonal Problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, Feb. 1, 1970, ISSN: 0040-1706. DOI: 10.1080/00401706.1970.10488634 Accessed: Sep. 22, 2026. [Online]. Available: https://doi.org/10.1080/00401706.1970.10488634

[12] R. Tibshirani, "Regression Shrinkage and Selection Via the Lasso," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, Jan. 1, 1996, ISSN: 0035-9246. DOI: 10.1111/j.2517-6161.1996.tb02080.x Accessed: Sep. 22, 2026. [Online]. Available: https://doi.org/10.1111/j.2517-6161.1996.tb02080.x

[13] F. Love, *NNTikZ: TikZ Diagrams for Deep Learning and Neural Networks*, 2024. Accessed: Oct. 15, 2026. [Online]. Available: https://github.com/fraserlove/nntikz

[14] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, ISSN: 1476-4687. DOI: 10.1038/323533a0 Accessed: Oct. 27, 2026. [Online]. Available: https://www.nature.com/articles/323533a0

[15] T. Hastie, R. Tibshirani, and J. H. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (Springer Series in Statistics), Second edition. New York, NY: Springer, 2017, ISBN: 978-0-387-84857-0.

[16] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, Y. W. Teh and M. Titterington, Eds., ser. Proceedings of Machine Learning Research, vol. 9, Chia Laguna Resort, Sardinia, Italy: PMLR, May 13–15, 2010, pp. 249–256. [Online]. Available: https://proceedings.mlr.press/v9/glorot10a.html

[17] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," in *2015 IEEE International Conference on Computer Vision (ICCV)*, Dec. 2015, pp. 1026–1034. DOI: 10.1109/ICCV.2015.123 Accessed: Nov. 10, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/7410480

[18] A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (Data Science / Machine Learning), Third edition. O'Reilly, 2023, 834 pp., ISBN: 978-1-0981-2597-4.

[19] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 17, 2020, ISSN: 0028-0836, 1476-4687. DOI: 10.1038/s41586-020-2649-2 Accessed: Aug. 23, 2026. [Online]. Available: https://www.nature.com/articles/s41586-020-2649-2

[20] C. Runge, "Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten," *Zeitschrift für Mathematik und Physik*, vol. 46, pp. 224–243, 1901.

[21] J. Bradbury et al., *JAX: Composable transformations of Python+NumPy programs*, version 0.3.13, 2018. [Online]. Available: http://github.com/jax-ml/jax

[22] A. M. Saxe, J. L. McClelland, and S. Ganguli. "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks." arXiv: 1312.6120 [cs.NE], Accessed: Nov. 2, 2026. [Online]. Available: http://arxiv.org/abs/1312.6120 pre-published.

APPENDIX

A. Use of AI/LLM tools

Large language models were used throughout this project, following the course guidelines[1], which also define the contribution levels below. All code was tested, all derivations were checked, and every claim in the report was read, understood and verified by the author.

1) Tools used:

- Claude Code (Claude Opus 5.5, accessed via the command-line interface, November 2026)
- ChatGPT (various models, accessed via the web, November 2026)

Claude Code was used as a coding assistant with access to the repository: to build the classification experiments and the logistic regression model, turn the author's activation and loss functions into classes, extend the training loop, write most of the tests, and fix linting and type errors. It was also used to review and restructure the report, and to give feedback on drafted sections. ChatGPT was sometimes used to ask for feedback on sections of the report and to discuss results.

2) Text writing and editing: Table VI gives the contribution level for each part of the report.

TABLE VI
LLM CONTRIBUTION TO TEXT WRITING AND EDITING (LEVELS 0–3).

Section	Level	Notes
Abstract	1	Language editing
Introduction	1	LLM suggested how to make the opening specific to this project and to shorten the outline; the text was rewritten by the author
Methods	1–2	LLM suggested the section order and pointed out errors in the backpropagation equations
Implementation	1–2	An early example was produced with an LLM and rewritten by the author; LLM feedback on length
Results and discussion	1–3	LLM suggested moving the MNIST description to section III and gave feedback on the presentation. In the final revision, an LLM drafted the short paragraph and caption for fig. 4 and the sentence on the depth of the classification grid
Conclusion	1	Language editing
Appendices	1	Language editing

3) Code generation and assistance: Table VII gives the level for each source file (levels 0–4). Functions with level 2 or above carry an `LLM-assisted` tag in their docstring.

4) Declared and undeclared use: The test comparing backpropagation with `jax.grad` was written by Claude Code, so it is declared in table VII and tagged in its docstring. Asking ChatGPT to explain a concept or discuss a result, and then writing about it in our own words, would not need to be declared.

B. Additional results

This appendix contains figures that support the discussion in section IV, but are not needed to follow it.

[1] https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/LLM_Usage_Declaration_Guidelines.md

TABLE VII
LLM CONTRIBUTION TO EACH CODE FILE AND NOTEBOOK (LEVELS 0–4). SOURCE FILES ARE GIVEN RELATIVE TO `src/fys_stk4155_p2/`.

File	Level	Description
`notebooks/regression.ipynb`	3	Refactoring code and markdown cells for readability and write the code for producing fig. 3. Parallelize the CV search for optimal hyperparameters.
`notebooks/classification.ipynb`	4	Wrote most of the notebook: the logistic regression baseline, the hyperparameter searches with cached results, the comparison of learning rate schedules, the figures, and the comparison with Scikit-Learn; moved the confusion matrices to the test set. Grids and experiment design chosen by the author.
`nn/activations.py`	3	Functions written independently, Claude made them into classes.
`nn/layers.py`	0	Written independently.
`nn/losses.py`	3	Functions written independently, Claude made them into classes.
`nn/network.py`	2	Completed the author's draft of learning rate schedules and early stopping in `FFNN.fit`.
`models/logistic.py`	4	Completely written by Claude from authors prompt.
`metrics.py`	0	Written independently, largely imported from Project 1.
`utils/logger.py`	0	Written independently.
`tests/test_network.py`	4	Wrote the test comparing backpropagation with `jax.grad`.
`tests/test_logistic.py`	4	Wrote most of the tests, and fixed two type errors flagged by mypy.
`tests/test_metrics.py`	4	Wrote most of the tests.
`tests/test_main.py`	4	Wrote the test.

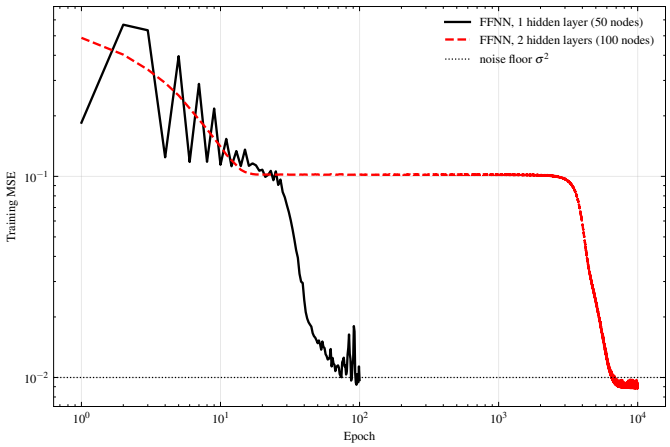


Fig. 13. Training MSE per epoch for the selected FFNNs: one hidden layer of 50 nodes (Adam, $\gamma = 0.1$, 100 epochs) and two hidden layers of 100 nodes (RMSprop, $\gamma = 10^{-4}$, 10000 epochs), both with a batch size of 32. The larger network stays at the mean prediction, with an MSE near the variance of the targets, for a large part of the training.

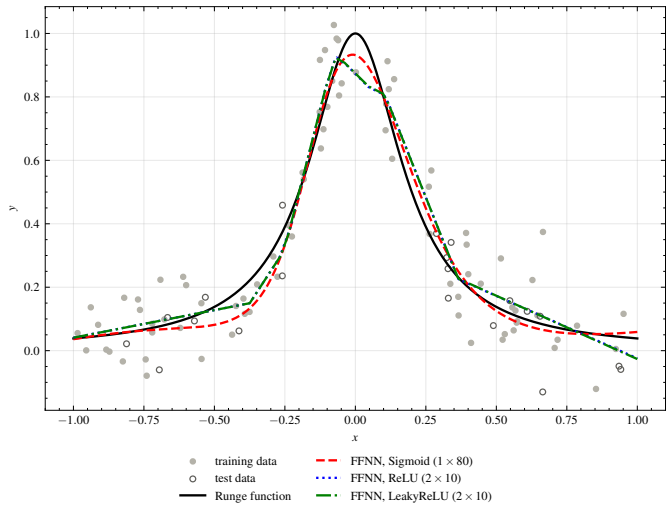


Fig. 14. Predictions of the best sigmoid, ReLU and Leaky ReLU networks from table II, together with the Runge function and the noisy training and test data. The ReLU and Leaky ReLU curves are nearly identical and consist of a few straight segments, while the sigmoid curve is smooth.

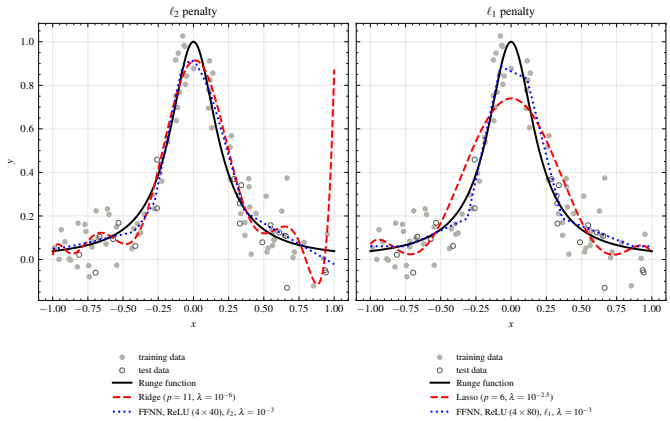


Fig. 15. The best ℓ_2 and ℓ_1 regularized networks, compared with the Ridge and Lasso models of Project 1, together with the Runge function and the noisy training and test data. Ridge oscillates near $x = 1$ and Lasso underfits the peak, while both networks follow the function closely.

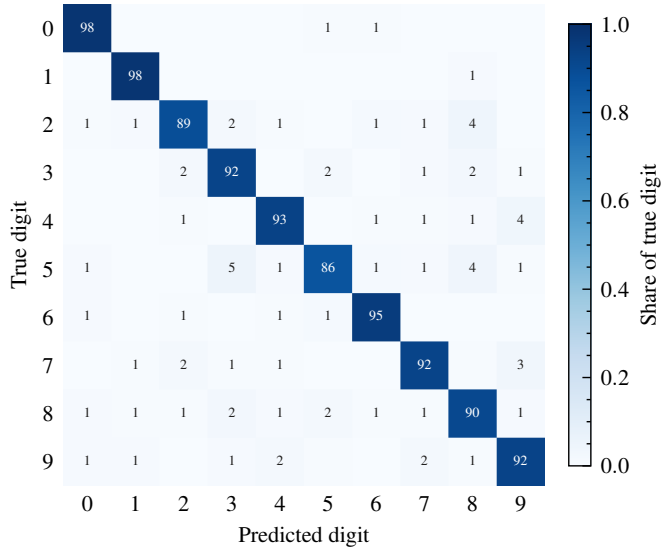


Fig. 16. Confusion matrix of logistic regression on the test set, normalized by row so that the diagonal is the recall of each digit (%). Most errors are between digits that share strokes, such as 3, 5 and 8.

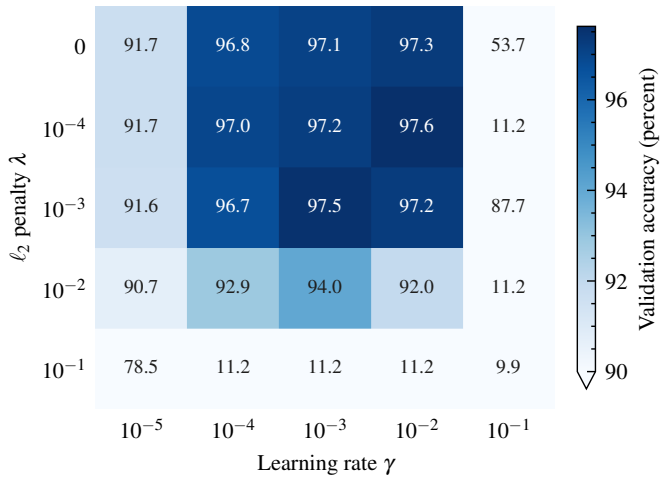


Fig. 17. Validation accuracy (%) of the selected architecture for each learning rate and ℓ_2 penalty. A small penalty helps slightly, while a large one, especially with a large learning rate, makes training fail.

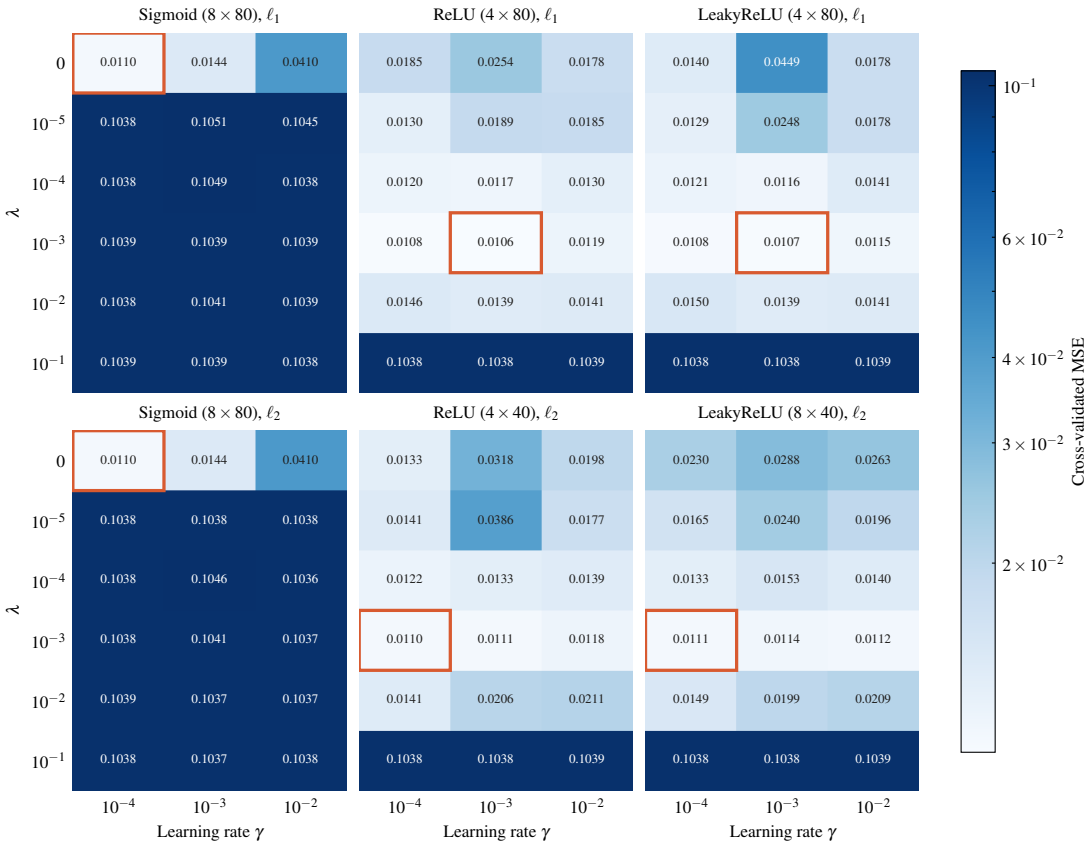


Fig. 18. Cross-validated MSE over the penalty λ and the learning rate γ, for the architecture of the best configuration of each activation and norm. The framed cell is the minimum. The ReLU family has its minimum at $\lambda = 10^{-3}$ for both norms, while every sigmoid network collapses to predicting the mean for all but the smallest penalties.